

5LSLO Machine Learning for signal processing - assignment 4

Gheorghiu Cristian Stefan*

Student number: 2036967

1 EXERCISE 4.6

1.1 4.6.a Requirement

Explain what the difference is between LISTA and neural proximal gradient descent. Why would the latter be a more suitable choice for accelerated MRI?

1.2 4.6.a Answer

LISTA and Neural PGD both unfold iterative optimization loops into neural networks but differ fundamentally in what they parameterize. LISTA dismantles the explicit connection to real-world acquisition physics. **As seen in 1 it replaces the mathematical measurement matrices with trainable weights to learn the physical mapping directly from data. Consequently, LISTA relies on a basic, mathematically rigid shrinking rule lacking spatial awareness.**

$$\begin{aligned} \mathbf{x}_{k+1} &= S_{\mu\lambda} \left((\mathbf{I} - \mu\mathbf{A}^\top \mathbf{A})\mathbf{x}_k + \mu\mathbf{A}^\top \mathbf{y} \right) \\ \rightarrow \mathbf{x}_{k+1} &= S_{\theta_k} (\mathbf{W}_{1,k}\mathbf{x}_k + \mathbf{W}_{2,k}\mathbf{y}) \end{aligned} \quad (1)$$

Works, such as [1] show that the Neural PGD takes the inverse approach by parameterizing the shrinking function instead of the physics. Within the context as defined by PGD the data consistency step remains untouched/hardcoded. **The proximal step is replaced by a spatially aware Convolutional Neural Network as shown in 2. Thus, the network learns a smart shrinking rule to recognize complex anatomical structures and remove aliasing artifacts.** Another architectural choice that might not be generic (but rather a specific choice authors of [1] deem necessary) but still counts as a difference translates to how PGD employs a weighted training scheme (β) that actively penalizes intermediate iterations if they deviate from the raw physical domain.

Neural PGD employs a weighted training scheme (β) that actively penalizes intermediate iterations if they deviate from the raw physical measurements. Because LISTA replaces physical matrices with learned weights, it typically relies on a standard end-to-end loss, lacking the ability to strictly enforce known physical consistency at every intermediate step.

$$\begin{aligned} \min_{\mathcal{W}, \alpha} & \beta \sum_{i=1}^N \ell(x_i, x_i^T) + (1 - \beta) \sum_{i=1}^N \sum_{t=1}^T \|y_i - \Phi x_i^t\|^2 \\ \text{s.t.} & \hat{x}_i^t = (\mathcal{P}_\psi \circ g)^t(\Phi^H y_i), \quad \forall i \in [N], t \in [T] \end{aligned} \quad (2)$$

Neural PGD is highly appropriate for accelerated MRI due to its computational feasibility. Hardcoding the Fourier transforms prevents the severe overfitting associated with learning massive global physics matrices. It frees the network's capacity to focus exclusively on learning complex non-linear human anatomy priors. Finally, enforcing strict data consistency guarantees the reconstructions remain physically plausible.

*e-mail: c.gheorghiu1@student.tue.nl

1.3 4.6.b Requirement

Implement neural proximal gradient descent for 5 unfolded iterations. As proximal operator, make use of the ConvNet you designed in exercise 5. Train it for at least 10 epochs and plot both the training loss and testing loss in terms of epochs or batches seen.

1.4 4.6.b Answer

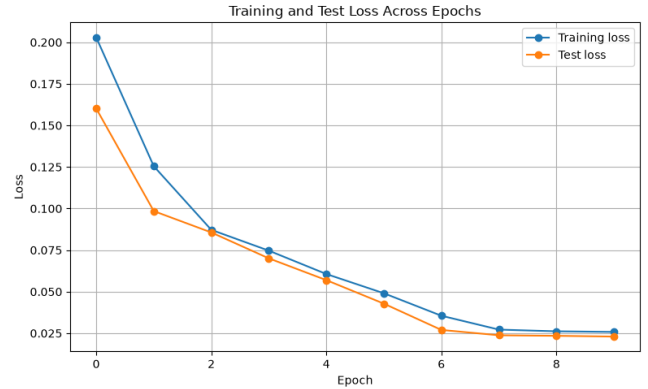


Figure 1: Training and Testing losses averaged over batches

Listing 1: Convolutional Proximal Operator (ConvISTA)

```
class ConvISTA(nn.Module):
    """
    Requirements: As a starting point for a deep
    learning solution,
    design an end-to-end convolutional neural
    network.
    As input it should take an initial
    reconstruction from a partial
    k-space measurement and output a guess for the
    final reconstruction.
    """
    def __init__(self, padding:str|int, stride:int,
                 kernel_size:int|tuple, width:list|int,
                 layers:int=4):
        super().__init__()
        self.NETWORK = nn.Sequential()
        self.width = width
        self.layers = layers

        self.padding = padding if type(padding) ==
            list else [padding]*layers
        self.kernel_size = kernel_size if type(
            kernel_size) == list else [kernel_size
            ]*layers

        for layer in range(layers):

            self.NETWORK.append(nn.Conv2d(
                in_channels=width[layer],
                out_channels=width[layer+1],
```

```

        kernel_size=self.ken_size[layer],
        padding=self.pading[layer]
    ))

    if layer != layers - 1:
        self.NETWORK.append(nn.Tanh())
    else:
        self.NETWORK.append(nn.ReLU())

def forward(self, x):
    # Convert absolute value to standard
    float32
    x_abs = torch.abs(x).to(torch.float32)
    x_final_rec = self.NETWORK(x_abs.unsqueeze(1))
    return x_final_rec

```

Design Rationale: The ConvISTA module employs a standard convolutional architecture consisting of four layers with configurable widths, kernel sizes, and strides. To ensure code conciseness and modularity, the network is built iteratively using `nn.Sequential`. Notably, earlier iterations of this architecture included Batch Normalization; however, enforcing zero-mean variance near the output layers inadvertently clamped the physical magnitude predictions, causing gradient collapse and severe visual artifacts (as observed in 2). Removing it entirely restored the network's stable learning capacity.

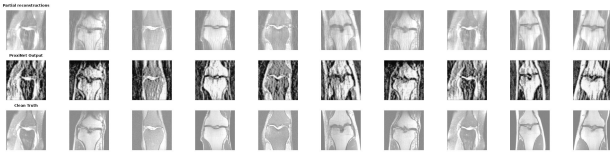


Figure 2: Batch-normalization compromised results

Listing 2: Convolutional Proximal Operator (ConvISTA)

```

class ProxNet(nn.Module):
    def __init__(self, conv_net: nn.Module,
                 num_iterations: int = 5):
        super().__init__()
        # 1. The Proximal Operator P_psi/ConvISTA instance
        self.prox_net = conv_net
        self.num_iterations: int = num_iterations

        # 2. Learnable step sizes (mu) for each iteration.
        # Initialized to 0.5. Use a Parameter to track gradients.
        self.mus = nn.Parameter(torch.full((
            num_iterations, ), 0.5))

    def forward(self, kspace: torch.Tensor, M:
                torch.Tensor):
        # 3.1 Initial zero-filled reconstruction (Phi^H y)
        x0_comp = get_accelerated_MRI(kspace * M)
        # 3.2 Initialize x_t as real-valued magnitude image
        x_t = torch.abs(x0_comp).to(torch.float32)

        for t in range(self.num_iterations):
            # 3.3 Enforce constraint: mu must be between 0 and 1
            mu = torch.clamp(self.mus[t], 0.0, 1.0)

```

```

# === 4: Data Consistency (The Physics 'g') ===
# 4.0 Extract phase to keep physics intact
c_phase = torch.angle(x0_comp) if t==0 else torch.angle(x_dc_comp)

# 4.1 Multiply real image by phase
x_t_comp = x_t.squeeze(1) * torch.exp(1j * c_phase)

# 4.2 Forward physics: Image to k-space (Phi)
k_t = get_k_space(x_t_comp)

# 4.3 Gradient step in k-space
k_dc = k_t - mu * M * (k_t - kspace * M)

# 4.4 Inverse physics: k-space back to Image domain
x_dc_comp = get_accelerated_MRI(k_dc)

# === 5. Proximal Operator (ConvNet) ===
# 5.1 Extract absolute magnitude for the network
x_dc_abs = torch.abs(x_dc_comp).to(torch.float32)

# 5.2 Pass through the CNN to denoise/inpaint
x_t = self.prox_net(x_dc_abs)

return x_t, x0_comp

```

Design Rationale: The ProxNet module directly translates the mathematical formulation of Neural Proximal Gradient Descent into an unrolled computational graph. Structural components, such as the ConvISTA proximal operator and the dynamically learnable step sizes (μ), are instantiated within the constructor. The iterative algorithm itself is executed sequentially within the forward pass, where the explicit data consistency step (applying mathematical MRI physics via the Fourier transform) strictly alternates with the spatially-aware CNN proximal step over exactly $K = 5$ unfolded iterations. [1] is central to the entire class development.

1.5 4.8.c Requirement

Repeat exercise 4b using your trained ProxNet:

1. b) Use your ISTA algorithm on 5 images from the test set. Show them in a figure consisting of 5 columns (1 per image) and 3 rows. Row 1 should contain the initial reconstruction from partial k-space, row 2 the reconstruction after ISTA, and row 3 the actual ground truth target. You will have to play with the values for μ , λ , and K . Explain what you see.

1.6 4.8.c Answer

Unlike classical ISTA, we cannot manually tune μ , λ , and K during inference. In Neural PGD, μ is an optimized learned parameter, λ is entirely replaced by the trained convolutional weights of the proximal network, and K is structurally locked to the 5 iterations the network was unrolled and trained for. Adopting a value for K other than 5 during inference time will result in an error due to the architectural mismatch between the blueprint saved along with the weights, and the actual architecture the network engages.

Without a shadow of a doubt, the reconstruction quality generated by ProxNet is highly comparable to the ground truth MRI

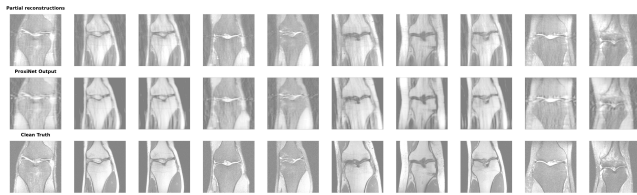


Figure 3: Three-levels comparison between Partial, Network Reconstructions and Ground Truth MRI

images, significantly outperforming the zero-filled partial reconstructions. The very first noticeable improvement (as seen in Fig 3 from left to right, comparing the partial reconstructions to the ProxiNet outputs) is that ProxiNet knows exactly the difference between the background and the anatomical elements. In the partial reconstructions, severe coherent aliasing artifacts (the repeating "ghosting" ripples) are pretty much all over the place, polluting the background. ProxiNet successfully eliminates this background noise. Technically, this positive effect is attributed to the Convolutional Neural Network acting as a spatially-aware proximal operator. Unlike classical math functions, the CNN learns the statistical distribution of human anatomy (spatial priors). It inherently recognizes that the air surrounding the knee should be uniform and zero, allowing it to aggressively filter out aliasing in the background without destroying the subject. With its current architecture, the ProxiNet is still somewhat able to capture fine-grained details (e.g., tendons, cartilage, or bone unifications), attaining a level of structural integrity found in the actual anatomy rather than the noisy partial reconstructions. However, there is still room for optimization to make these ultra-fine details even finer. If we were to build a network with greater depth - specifically by increasing the number of unfolded iterations ($K > 5$) the network would be able to depict these details much better. A higher K allows the algorithm to alternate more times between enforcing raw physics (Data Consistency) and removing noise (the CNN Proximal step), allowing the network to iteratively refine and sharpen the micro-structures of the knee. Furthermore, beyond simply adding layers, we could tweak the training objective itself to force the network to care more about high-frequency detail. Specifically, we could adjust the weighting factor (β) in the Neural PGD loss formulation. The β term acts as a tuning dial that balances how much the network penalizes intermediate physical errors versus the final visual output error.

1.7 Requirement 4.8.d

Repeat exercise 4c using your trained ConvNet

1. c) To get a numerical estimate for the performance of ISTA, run it on the entire test set and report the resulting mean squared error (mse)

1.8 Answer 4.8.d

Table 1: Numerical Performance Estimate of the Trained ProxiNet

Model Architecture	Test Set MSE
ProxiNet (Neural PGD, $K = 5$)	0.027541

1.9 Requirement 4.8.e

A direct comparison between your ProxiNet and ConvNet might not be fair, why? Can you think of a way to make the comparison fairer? Implement this and compare mse again

1.10 Requirement 4.8.e

There are two aspects worth considering and discussing in order to properly address the question: **Information Asymmetry** ("The Physics") advantages ProxiNet as this makes for the use of the exact physical measurements (raw k -space) and the sampling mask (M) multiple times to correct its predictions via the Data Consistency steps. Judged from this standpoint, the standalone ConvNet is "blind" - as it only sees the initial aliased image and has no mechanism to verify its output against the actual scanner data. The second aspect worth factoring in is **Computational Depth**: ProxiNet runs the image through the convolutional block 5 times sequentially ($K = 5$). The standalone ConvNet only processes the image once, meaning it possesses only a fraction of the computational depth and receptive field.

To "level the field" one can can apply a Data Consistency (DC) post-processing step to the ConvNet's output during inference. As shown in table 2 this method will yield even greater results.

Table 2: Fairness Comparison: Standalone ConvNet vs. Physics-Injected ConvNet

Model Configuration	Test Set MSE
Standalone ConvNet (Unfair Baseline)	0.269894
ConvNet + Data Consistency (Fairer)	0.015410

REFERENCES

- [1] Jihun Yun, Aurélie C. Lozano, and Eunho Yang. "Adaptive Proximal Gradient Methods for Structured Neural Networks". In: *KAIST, IBM T.J. Watson Research Center, AITRICS* (n.d.).